Advanced Computer Systems

Workshop 1

Dr. Shounak Chakraborty,
Assistant Professor,
Dept. of Computer Science,
Durham University, UK

Oct 13, 2026

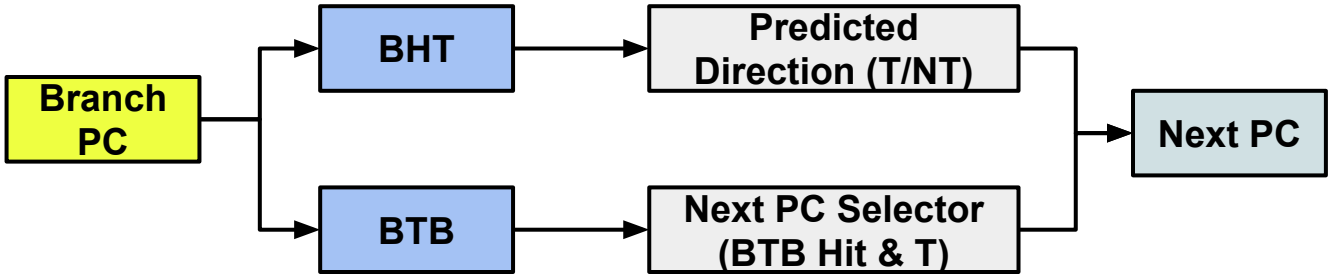
Durham University

COMP3791

Implementing Branch Prediction

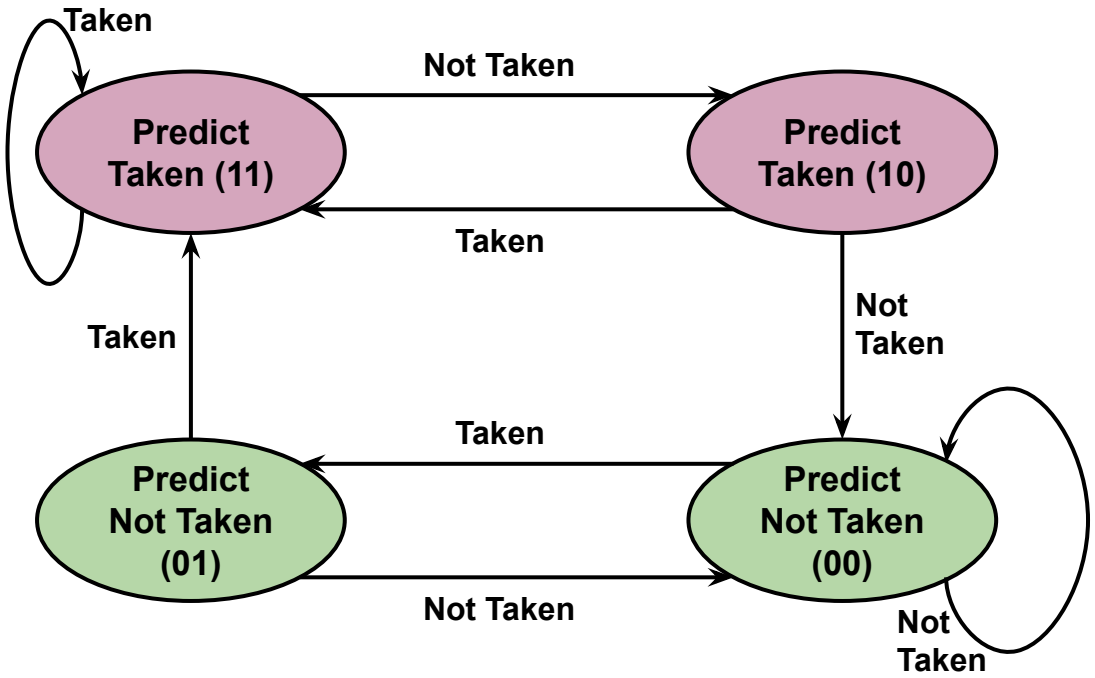
The Branch Prediction Pipeline

- **The Front-End Challenge:** In cycle IF, we must know two things before decode:
  - **Direction:** Will the branch be Taken or Not-Taken? (Provided by the BHT / 2-bit counters)
  - **Target PC:** If Taken, where are we jumping to? (Provided by the BTB)
- **Hit vs. Miss Cases:**
  - **BTB Hit + Predict Taken:** Redirect next fetch address to BTB Target PC
  - **BTB Hit + Predict Not Taken:** Fetch fall-through PC (PC + 4)
  - **BTB Miss:** Fetch fall-through PC (PC + 4); if the branch is later resolved as Taken in EX, incur a pipeline redirect penalty



2-bit Saturating Counter Implementation

- **Finite State Machine & Update Rules**
  - **Index Calculation: bht_index = (branch_pc >> 2) & ((1 << bht_index_bits) - 1)**
  - **Prediction Logic: Output Taken if MSB is 1 (state >= 2), else Not Taken (state < 2)**
  - **State Update Transitions:**
    - On actual `Taken`: `state = min(state + 1, 3)`
    - On actual `Not Taken`: `state = max(state - 1, 0)`
  - **The Hysteresis Benefit:** Eliminates the classic single-bit ping-pong anomaly on loop exits (T … T, NT)



Direct-Mapped BTB Implementation

- **Cache-Like Structure for Target Addresses**
  - **Address Decomposition (32-bit Architecture):**
    - Byte offset: 2 bits (instructions are 4-byte aligned; bits [1:0] are always zero)
    - Index bits (K): [K+1 : 2], indexing into 2^K entries
    - Tag bits: Remaining upper bits [31 : K+2]

```
struct BTBEntry {
    bool valid;             // 1 bit
    uint32_t tag;           // 32 - (K + 2) bits
    uint32_t target_pc;     // 32 bits
};
```

BTB Entry Data Structure (C++ Layout)

- **Lookup & Update Sequence:**
  - **Lookup (`pc`):** Compute `index` and `tag`. If `entry.valid && entry.tag == tag`, return `(Hit, entry.target_pc)`. Else return `(Miss, pc + 4)`
  - **Update (`pc`, `target_pc`, `taken`):** On execution, if branch is Taken, install/update entry:
    - `entry.valid = true`, `entry.tag = computed_tag`, `entry.target_pc = actual_target`

Trace Evaluation & Aliasing Behaviour

- **Empirical Evaluation across Branch Behaviours**
  - **Loop Workload Traces:**
    - **Biased Loop:** 8-iteration loop: T, T, T, T, T, T, T, NT
      - *1-bit Predictor:* Mispredicts twice (last iteration + next entry into loop)
      - *2-bit Predictor:* Drops to 1 misprediction per loop execution once steady state is reached (stays at Weakly Taken on exit)
    - **Alternating Pattern:** T, NT, T, NT, T, NT...
      - 2-bit counter fails catastrophically (~ 0% accuracy)
      - *Fix:* Correlating / Two-level predictor tracking global history (Gshare)
  - **Constructive vs. Destructive Aliasing:**
    - Multiple branch PCs index into the same table slot when index bits are constrained
    - Different branches with conflicting tendencies thrash shared counter states

Hardware Cost and Storage Overhead

- **Quantifying Real-Silicon Budget**
  - **Concrete Sizing Scenario**
    - 32-bit RISC processor with 4-byte instruction alignment
    - 1024-entry (2^{10}) direct-mapped BTB
    - 1024-entry (2^{10}) BHT (2-bit saturating counters)
  - **BTB Storage Breakdown**
    - Index bits: 10 bits → Offset bits: 2 bits
    - Tag size: 32 - (10 + 2) = 20 bits
    - Entry width: 1 (valid bit) + 20 (tag) + 32 (target PC) = 53 bits/entry
    - Total BTB Storage: 1024 * 53 bits = 54,272 bits or 6.625 KiB
  - **BHT Storage Breakdown**
    - Total BHT Storage: 1024 * 2 bits = 2,048 bits = 256 Bytes

Key Takeaway: Direction tables (BHT) are exceptionally cheap per entry; target buffers (BTB) require significantly more silicon area due to 32/64-bit target storage and tag verification

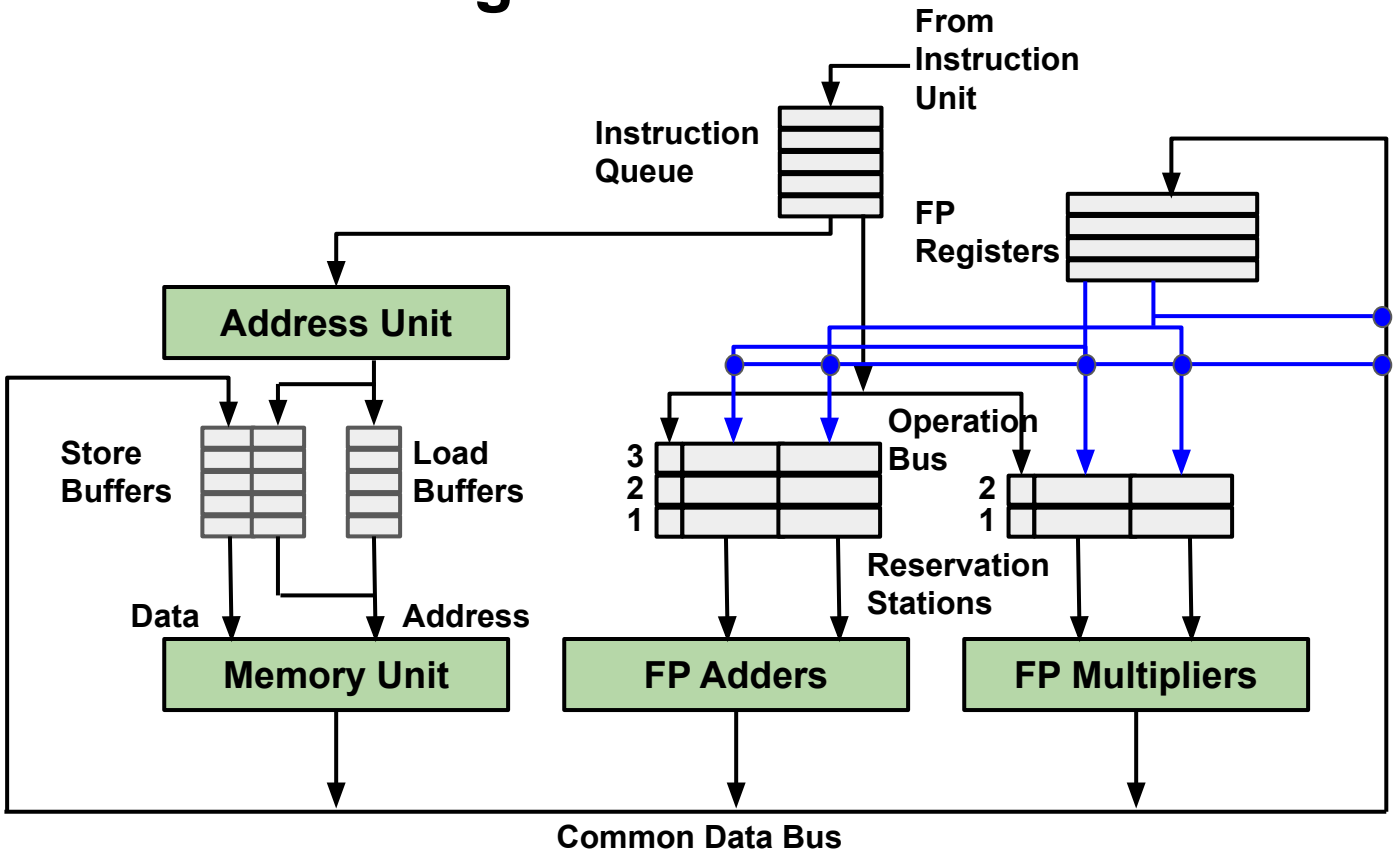
Out-of-Order Execution

Issues in Out-of-Order Execution

- **Dynamic Scheduling may generate *imprecise exception*:**
  - **The pipeline may have *already completed* instructions that are *later* in program order than the instruction causing the exception.**
  - **The pipeline may have *not yet completed* some instructions that are *earlier* in program order than the instruction causing the exception**

- **To allow out-of-order execution, we essentially split the ID pipe stage of our MIPS pipeline into two stages:**
  - **Issue—Decode instructions, check for structural hazards**
  - **Read operands—Wait until no data hazards, then read operands**

Tomasulo's Algorithm



3 Stages:

- Issue
- Execute
- Write Results

- R. M. Tomasulo, "An Efficient Algorithm for Exploiting Multiple Arithmetic Units," in *IBM Journal of Research and Development*, vol. 11, no. 1, pp. 25-33, Jan. 1967, doi: 10.1147/rd.111.0025.

Tomasulo's Algorithm (cont…)

- Op— Operation to perform in the unit (e.g., + or –)
- Qj, Qk— Reservation stations producing source registers
- Vj, Vk— Value of Source operands
- Rj, Rk— Flags indicating when Vj, Vk are ready
- Busy— Indicates reservation station and FU is busy
- Register result status— Indicates which functional unit will write each register, if one exists. Blank when no pending instructions that will write that register

Three Stages of Tomasulo's Algorithm

1. **Issue**—get instruction from FP Op Queue
   - If reservation station free, the scoreboard issues instr & sends operands (renames registers)
2. **Execution**—operate on operands (EX)
   - When both operands ready then execute; if not ready, watch CDB for result
3. **Write result**—finish execution (WB)
   - Write on Common Data Bus to all awaiting units; mark reservation station available

Tomasulo Example Cycle 0

Instruction status

Instruction		j	k	Issue	Execution complete	Write Result
LD	F6	34+	R2			
LD	F2	45+	R3			
MULT	F0	F2	F4			
SUBD	F8	F6	F2			
DIVD	F10	F0	F6			
ADDD	F6	F8	F2			

	Busy	Address
Load1	No	
Load2	No	
Load3	No	

Reservation Stations

Time	Name	Busy	Op	S1 Vj	S2 Vk	RS for j Qj	RS for k Qk
0	Add1	No					
0	Add2	No					
	Add3	No					
0	Mult1	No					
0	Mult2	No					

Register result status

Clock	F0	F2	F4	F6	F8	F10	F12	...	F30
0	FU								

Tomasulo Example Cycle 1

Instruction status

Instruction		j	k	Issue	Execution complete	Write Result
LD	F6	34+	R2	1		
LD	F2	45+	R3			
MULT	F0	F2	F4			
SUBD	F8	F6	F2			
DIVD	F10	F0	F6			
ADDD	F6	F8	F2			

	Busy	Address
Load1	Yes	34+R2
Load2	No	
Load3	No	

Reservation Stations

Time	Name	Busy	Op	S1 Vj	S2 Vk	RS for j Qj	RS for k Qk
0	Add1	No					
0	Add2	No					
	Add3	No					
0	Mult1	No					
0	Mult2	No					

Register result status

Clock		F0	F2	F4	F6	F8	F10	F12	...	F30
1	FU				Load1					

Tomasulo Example Cycle 2

Instruction status

Instruction		j	k	Issue	Execution complete	Write Result
LD	F6	34+	R2	1		
LD	F2	45+	R3	2		
MULT	F0	F2	F4			
SUBD	F8	F6	F2			
DIVD	F10	F0	F6			
ADDD	F6	F8	F2			

	Busy	Address
Load1	Yes	34+R2
Load2	Yes	45+R3
Load3	No	

Reservation Stations

Time	Name	Busy	Op	S1 Vj	S2 Vk	RS for j Qj	RS for k Qk
0	Add1	No					
0	Add2	No					
	Add3	No					
0	Mult1	No					
0	Mult2	No					

Register result status

Clock		F0	F2	F4	F6	F8	F10	F12	...	F30
2	FU		Load2		Load1					

Tomasulo Example Cycle 3

Instruction status

Instruction		j	k	Issue	Execution complete	Write Result
LD	F6	34+	R2	1	3	
LD	F2	45+	R3	2		
MULT	F0	F2	F4	3		
SUBD	F8	F6	F2			
DIVD	F10	F0	F6			
ADDD	F6	F8	F2			

	Busy	Address
Load1	Yes	34+R2
Load2	Yes	45+R3
Load3	No	

Reservation Stations

Time	Name	Busy	Op	S1 Vj	S2 Vk	RS for j Qj	RS for k Qk
0	Add1	No					
0	Add2	No					
	Add3	No					
0	Mult1	Yes	MULTD		R(F4)	Load2	
0	Mult2	No					

Register result status

Clock		F0	F2	F4	F6	F8	F10	F12	...	F30
3	FU	Mult1	Load2		Load1					

Tomasulo Example Cycle 4

Instruction status

Instruction		j	k	Issue	Execution complete	Write Result
LD	F6	34+	R2	1	3	4
LD	F2	45+	R3	2		
MULT	F0	F2	F4	3		
SUBD	F8	F6	F2	4		
DIVD	F10	F0	F6			
ADDD	F6	F8	F2			

	Busy	Address
Load1	No	
Load2	Yes	45+R3
Load3	No	

Reservation Stations

Time	Name	Busy	Op	S1 Vj	S2 Vk	RS for j Qj	RS for k Qk
0	Add1	Yes	SUBD	M(34+R2)			Load2
0	Add2	No					
	Add3	No					
0	Mult1	Yes	MULTD		R(F4)	Load2	
0	Mult2	No					

Register result status

Clock		F0	F2	F4	F6	F8	F10	F12	...	F30
4	FU	Mult1	Load2		M(34+R2)	Add1				

Tomasulo Example Cycle 5

Instruction status

Instruction		j	k	Issue	Execution complete	Write Result
LD	F6	34+	R2	1	3	4
LD	F2	45+	R3	2	5	
MULT	F0	F2	F4	3		
SUBD	F8	F6	F2	4		
DIVD	F10	F0	F6	5		
ADDD	F6	F8	F2			

	Busy	Address
Load1	No	
Load2	Yes	45+R3
Load3	No	

Reservation Stations

Time	Name	Busy	Op	S1 Vj	S2 Vk	RS for j Qj	RS for k Qk
0	Add1	Yes	SUBD	M(34+R2)			Load2
0	Add2	No					
	Add3	No					
0	Mult1	Yes	MULTD		R(F4)	Load2	
0	Mult2	Yes	DIVD		M(34+R2)	Mult1	

Register result status

Clock		F0	F2	F4	F6	F8	F10	F12	...	F30
5	FU	Mult1	Load2		M(34+R2)	Add1	Mult2			

Tomasulo Example Cycle 6

Instruction status

Instruction		j	k	Issue	Execution complete	Write Result
LD	F6	34+	R2	1	3	4
LD	F2	45+	R3	2	5	6
MULT	F0	F2	F4	3		
SUBD	F8	F6	F2	4		
DIVD	F10	F0	F6	5		
ADDD	F6	F8	F2	6		

	Busy	Address
Load1	No	
Load2	No	
Load3	No	

Reservation Stations

Time	Name	Busy	Op	S1 Vj	S2 Vk	RS for j Qj	RS for k Qk
2	Add1	Yes	SUBD	M(34+R2)	M(45+R3)		
0	Add2	Yes	ADDD		M(45+R3)	Add1	
	Add3	No					
10	Mult1	Yes	MULTD	M(45+R3)	R(F4)		
0	Mult2	Yes	DIVD		M(34+R2)	Mult1	

Register result status

Clock		F0	F2	F4	F6	F8	F10	F12	...	F30
6	FU	Mult1	M(45+R3)		Add2	Add1	Mult2			

Tomasulo Example Cycle 7

Instruction status

Instruction		j	k	Issue	Execution complete	Write Result
LD	F6	34+	R2	1	3	4
LD	F2	45+	R3	2	5	6
MULT	F0	F2	F4	3		
SUBD	F8	F6	F2	4		
DIVD	F10	F0	F6	5		
ADDD	F6	F8	F2	6		

	Busy	Address
Load1	No	
Load2	No	
Load3	No	

Reservation Stations

Time	Name	Busy	Op	S1 Vj	S2 Vk	RS for j Qj	RS for k Qk
1	Add1	Yes	SUBD	M(34+R2)	M(45+R3)		
0	Add2	Yes	ADDD		M(45+R3)	Add1	
	Add3	No					
9	Mult1	Yes	MULTD	M(45+R3)	R(F4)		
0	Mult2	Yes	DIVD		M(34+R2)	Mult1	

Register result status

Clock		F0	F2	F4	F6	F8	F10	F12	...	F30
7	FU	Mult1	M(45+R3)		Add2	Add1	Mult2			

Tomasulo Example Cycle 8

Instruction status

Instruction		j	k	Issue	Execution complete	Write Result
LD	F6	34+	R2	1	3	4
LD	F2	45+	R3	2	5	6
MULT	F0	F2	F4	3		
SUBD	F8	F6	F2	4	8	
DIVD	F10	F0	F6	5		
ADDD	F6	F8	F2	6		

	Busy	Address
Load1	No	
Load2	No	
Load3	No	

Reservation Stations

Time	Name	Busy	Op	S1 Vj	S2 Vk	RS for j Qj	RS for k Qk
0	Add1	Yes	SUBD	M(34+R2)	M(45+R3)		
0	Add2	Yes	ADDD		M(45+R3)	Add1	
	Add3	No					
8	Mult1	Yes	MULTD	M(45+R3)	R(F4)		
0	Mult2	Yes	DIVD		M(34+R2)	Mult1	

Register result status

Clock		F0	F2	F4	F6	F8	F10	F12	...	F30
8	FU	Mult1	M(45+R3)		Add2	Add1	Mult2			

Tomasulo Example Cycle 9

Instruction status

Instruction		j	k	Issue	Execution complete	Write Result
LD	F6	34+	R2	1	3	4
LD	F2	45+	R3	2	5	6
MULT	F0	F2	F4	3		
SUBD	F8	F6	F2	4	8	9
DIVD	F10	F0	F6	5		
ADDD	F6	F8	F2	6		

	Busy	Address
Load1	No	
Load2	No	
Load3	No	

Reservation Stations

Time	Name	Busy	Op	S1 Vj	S2 Vk	RS for j Qj	RS for k Qk
0	Add1	No					
0	Add2	Yes	ADDD	M()−M()	M(45+R3)		
	Add3	No					
7	Mult1	Yes	MULTD	M(45+R3)	R(F4)		
0	Mult2	Yes	DIVD		M(34+R2)	Mult1	

Register result status

Clock		F0	F2	F4	F6	F8	F10	F12	...	F30
9	FU	Mult1	M(45+R3)		Add2	M()−M()	Mult2			

Tomasulo Example Cycle 10

Instruction status

Instruction		j	k	Issue	Execution complete	Write Result
LD	F6	34+	R2	1	3	4
LD	F2	45+	R3	2	5	6
MULT	F0	F2	F4	3		
SUBD	F8	F6	F2	4	8	9
DIVD	F10	F0	F6	5		
ADDD	F6	F8	F2	6		

	Busy	Address
Load1	No	
Load2	No	
Load3	No	

Reservation Stations

Time	Name	Busy	Op	S1 Vj	S2 Vk	RS for j Qj	RS for k Qk
0	Add1	No					
2	Add2	Yes	ADDD	M()−M()	M(45+R3)		
	Add3	No					
7	Mult1	Yes	MULTD	M(45+R3)	R(F4)		
0	Mult2	Yes	DIVD		M(34+R2)	Mult1	

Register result status

Clock		F0	F2	F4	F6	F8	F10	F12	...	F30
10	FU	Mult1	M(45+R3)		Add2	M()−M()	Mult2			

Tomasulo Example Cycle 11

Instruction status

Instruction		j	k	Issue	Execution complete	Write Result
LD	F6	34+	R2	1	3	4
LD	F2	45+	R3	2	5	6
MULT	F0	F2	F4	3		
SUBD	F8	F6	F2	4	8	9
DIVD	F10	F0	F6	5		
ADDD	F6	F8	F2	6		

	Busy	Address
Load1	No	
Load2	No	
Load3	No	

Reservation Stations

Time	Name	Busy	Op	S1 Vj	S2 Vk	RS for j Qj	RS for k Qk
0	Add1	No					
1	Add2	Yes	ADDD	M()−M()	M(45+R3)		
	Add3	No					
5	Mult1	Yes	MULTD	M(45+R3)	R(F4)		
0	Mult2	Yes	DIVD		M(34+R2)	Mult1	

Register result status

Clock		F0	F2	F4	F6	F8	F10	F12	...	F30
11	FU	Mult1	M(45+R3)		Add2	M()−M()	Mult2			

Tomasulo Example Cycle 12

Instruction status

Instruction		j	k	Issue	Execution complete	Write Result
LD	F6	34+	R2	1	3	4
LD	F2	45+	R3	2	5	6
MULT	F0	F2	F4	3		
SUBD	F8	F6	F2	4	8	9
DIVD	F10	F0	F6	5		
ADDD	F6	F8	F2	6	12	

	Busy	Address
Load1	No	
Load2	No	
Load3	No	

Reservation Stations

Time	Name	Busy	Op	S1 Vj	S2 Vk	RS for j Qj	RS for k Qk
0	Add1	No					
0	Add2	Yes	ADDD	M()−M()	M(45+R3)		
	Add3	No					
4	Mult1	Yes	MULTD	M(45+R3)	R(F4)		
0	Mult2	Yes	DIVD		M(34+R2)	Mult1	

Register result status

Clock		F0	F2	F4	F6	F8	F10	F12	...	F30
12	FU	Mult1	M(45+R3)		Add2	M()−M()	Mult2			

Tomasulo Example Cycle 13

Instruction status

Instruction		j	k	Issue	Execution complete	Write Result
LD	F6	34+	R2	1	3	4
LD	F2	45+	R3	2	5	6
MULT	F0	F2	F4	3		
SUBD	F8	F6	F2	4	8	9
DIVD	F10	F0	F6	5		
ADDD	F6	F8	F2	6	12	13

	Busy	Address
Load1	No	
Load2	No	
Load3	No	

Reservation Stations

Time	Name	Busy	Op	S1 Vj	S2 Vk	RS for j Qj	RS for k Qk
0	Add1	No					
0	Add2	No					
	Add3	No					
3	Mult1	Yes	MULTD	M(45+R3)	R(F4)		
0	Mult2	Yes	DIVD		M(34+R2)	Mult1	

Register result status

Clock		F0	F2	F4	F6	F8	F10	F12	...	F30
13	FU	Mult1	M(45+R3)		(M−M)+M()	M()−M()	Mult2			

Tomasulo Example Cycle 14

Instruction status

Instruction		j	k	Issue	Execution complete	Write Result
LD	F6	34+	R2	1	3	4
LD	F2	45+	R3	2	5	6
MULT	F0	F2	F4	3		
SUBD	F8	F6	F2	4	8	9
DIVD	F10	F0	F6	5		
ADDD	F6	F8	F2	6	12	13

	Busy	Address
Load1	No	
Load2	No	
Load3	No	

Reservation Stations

Time	Name	Busy	Op	S1 Vj	S2 Vk	RS for j Qj	RS for k Qk
0	Add1	No					
0	Add2	No					
	Add3	No					
2	Mult1	Yes	MULTD	M(45+R3)	R(F4)		
0	Mult2	Yes	DIVD		M(34+R2)	Mult1	

Register result status

Clock		F0	F2	F4	F6	F8	F10	F12	...	F30
14	FU	Mult1	M(45+R3)		(M−M)+M()	M()−M()	Mult2			

Tomasulo Example Cycle 15

Instruction status

Instruction		j	k	Issue	Execution complete	Write Result
LD	F6	34+	R2	1	3	4
LD	F2	45+	R3	2	5	6
MULT	F0	F2	F4	3		
SUBD	F8	F6	F2	4	8	9
DIVD	F10	F0	F6	5		
ADDD	F6	F8	F2	6	12	13

	Busy	Address
Load1	No	
Load2	No	
Load3	No	

Reservation Stations

Time	Name	Busy	Op	S1 Vj	S2 Vk	RS for j Qj	RS for k Qk
0	Add1	No					
0	Add2	No					
	Add3	No					
1	Mult1	Yes	MULTD	M(45+R3)	R(F4)		
0	Mult2	Yes	DIVD		M(34+R2)	Mult1	

Register result status

Clock		F0	F2	F4	F6	F8	F10	F12	...	F30
15	FU	Mult1	M(45+R3)		(M−M)+M()	M()−M()	Mult2			

Tomasulo Example Cycle 16

Instruction status

Instruction		j	k	Issue	Execution complete	Write Result
LD	F6	34+	R2	1	3	4
LD	F2	45+	R3	2	5	6
MULT	F0	F2	F4	3	16	
SUBD	F8	F6	F2	4	8	9
DIVD	F10	F0	F6	5		
ADDD	F6	F8	F2	6	12	13

	Busy	Address
Load1	No	
Load2	No	
Load3	No	

Reservation Stations

Time	Name	Busy	Op	S1 Vj	S2 Vk	RS for j Qj	RS for k Qk
0	Add1	No					
0	Add2	No					
	Add3	No					
0	Mult1	Yes	MULTD	M(45+R3)	R(F4)		
0	Mult2	Yes	DIVD		M(34+R2)	Mult1	

Register result status

Clock		F0	F2	F4	F6	F8	F10	F12	...	F30
16	FU	Mult1	M(45+R3)		(M−M)+M()	M()−M()	Mult2			

Tomasulo Example Cycle 17

Instruction status

Instruction		j	k	Issue	Execution complete	Write Result
LD	F6	34+	R2	1	3	4
LD	F2	45+	R3	2	5	6
MULT	F0	F2	F4	3	16	17
SUBD	F8	F6	F2	4	8	9
DIVD	F10	F0	F6	5		
ADDD	F6	F8	F2	6	12	13

	Busy	Address
Load1	No	
Load2	No	
Load3	No	

Reservation Stations

Time	Name	Busy	Op	S1 Vj	S2 Vk	RS for j Qj	RS for k Qk
0	Add1	No					
0	Add2	No					
	Add3	No					
0	Mult1	No					
0	Mult2	Yes	DIVD	M*F4	M(34+R2)		

Register result status

Clock		F0	F2	F4	F6	F8	F10	F12	...	F30
17	FU	M*F4	M(45+R3)		(M−M)+M()	M()−M()	Mult2			

Tomasulo Example Cycle 18

Instruction status

Instruction		j	k	Issue	Execution complete	Write Result
LD	F6	34+	R2	1	3	4
LD	F2	45+	R3	2	5	6
MULT	F0	F2	F4	3	16	17
SUBD	F8	F6	F2	4	8	9
DIVD	F10	F0	F6	5		
ADDD	F6	F8	F2	6	12	13

	Busy	Address
Load1	No	
Load2	No	
Load3	No	

Reservation Stations

Time	Name	Busy	Op	S1 Vj	S2 Vk	RS for j Qj	RS for k Qk
0	Add1	No					
0	Add2	No					
	Add3	No					
0	Mult1	No					
40	Mult2	Yes	DIVD	M*F4	M(34+R2)		

Register result status

Clock		F0	F2	F4	F6	F8	F10	F12	...	F30
18	FU	M*F4	M(45+R3)		(M−M)+M()	M()−M()	Mult2			

Tomasulo Example Cycle 57

Instruction status

Instruction		j	k	Issue	Execution complete	Write Result
LD	F6	34+	R2	1	3	4
LD	F2	45+	R3	2	5	6
MULT	F0	F2	F4	3	16	17
SUBD	F8	F6	F2	4	8	9
DIVD	F10	F0	F6	5		
ADDD	F6	F8	F2	6	12	13

	Busy	Address
Load1	No	
Load2	No	
Load3	No	

Reservation Stations

Time	Name	Busy	Op	S1 Vj	S2 Vk	RS for j Qj	RS for k Qk
0	Add1	No					
0	Add2	No					
	Add3	No					
0	Mult1	No					
1	Mult2	Yes	DIVD	M*F4	M(34+R2)		

Register result status

Clock		F0	F2	F4	F6	F8	F10	F12	...	F30
57	FU	M*F4	M(45+R3)		(M−M)+M()	M()−M()	Mult2			

Tomasulo Example Cycle 57

Instruction status

Instruction		j	k	Issue	Execution complete	Write Result
LD	F6	34+	R2	1	3	4
LD	F2	45+	R3	2	5	6
MULT	F0	F2	F4	3	16	17
SUBD	F8	F6	F2	4	8	9
DIVD	F10	F0	F6	5	**57**	
ADDD	F6	F8	F2	6	12	13

	Busy	Address
Load1	No	
Load2	No	
Load3	No	

Reservation Stations

Time	Name	Busy	Op	S1 Vj	S2 Vk	RS for j Qj	RS for k Qk
0	Add1	No					
0	Add2	No					
	Add3	No					
0	Mult1	No					
0	Mult2	Yes	DIVD	M*F4	M(34+R2)		

Register result status

Clock		F0	F2	F4	F6	F8	F10	F12	...	F30
57	FU	M*F4	M(45+R3)		(M−M)+M()	M()−M()	Mult2			

Tomasulo Example Cycle 58

Instruction status

Instruction		j	k	Issue	Execution complete	Write Result
LD	F6	34+	R2	1	3	4
LD	F2	45+	R3	2	5	6
MULT	F0	F2	F4	3	16	17
SUBD	F8	F6	F2	4	8	9
DIVD	F10	F0	F6	5	**57**	**58**
ADDD	F6	F8	F2	6	12	13

	Busy	Address
Load1	No	
Load2	No	
Load3	No	

Reservation Stations

Time	Name	Busy	Op	S1 Vj	S2 Vk	RS for j Qj	RS for k Qk
0	Add1	No					
0	Add2	No					
	Add3	No					
0	Mult1	No					
0	Mult2	No					

Register result status

Clock		F0	F2	F4	F6	F8	F10	F12	...	F30
58	FU	M*F4	M(45+R3)		(M−M)+M()	M()−M()	M*F4/M			

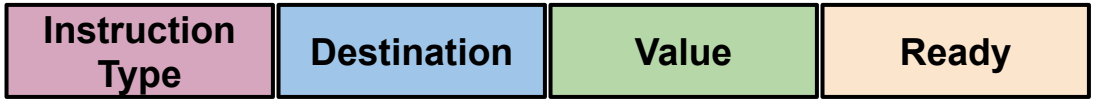
Summary of Tomasulo's Algorithm

- Prevents Register as bottleneck
- Avoids WAR, WAW hazards of Scoreboard
- Allows loop unrolling in HW
- Not limited to basic blocks (provided branch prediction)
- Lasting Contributions
  - Dynamic scheduling
  - Register renaming
  - Load/store disambiguation

Hardware Speculation

- Branches: a bottleneck for higher ILP in Tomasulo's Algorithm
  - Need solutions for control dependencies
- Hardware speculation can be a solution
  - Fetch, Issue and Execute work well if speculation is correct
  - Need mechanism to handle when speculation fails
- Hardware speculation combines three ideas-
  - Dynamic Branch Prediction
  - Speculation to execute instructions before resolving branch
    - Mechanism should be integrated to handle mis-speculation
  - Dynamic Scheduling to deal with the scheduling of different combinations of basic blocks
- Resembles Data Flow Execution, right?
- Execute in Out-of-Order, but commit In-Order
  - Hence, introduced Re-Order Buffer (ROB): Similar to store buffers in Tomasulo

Re-Order Buffer



An entry of ROB

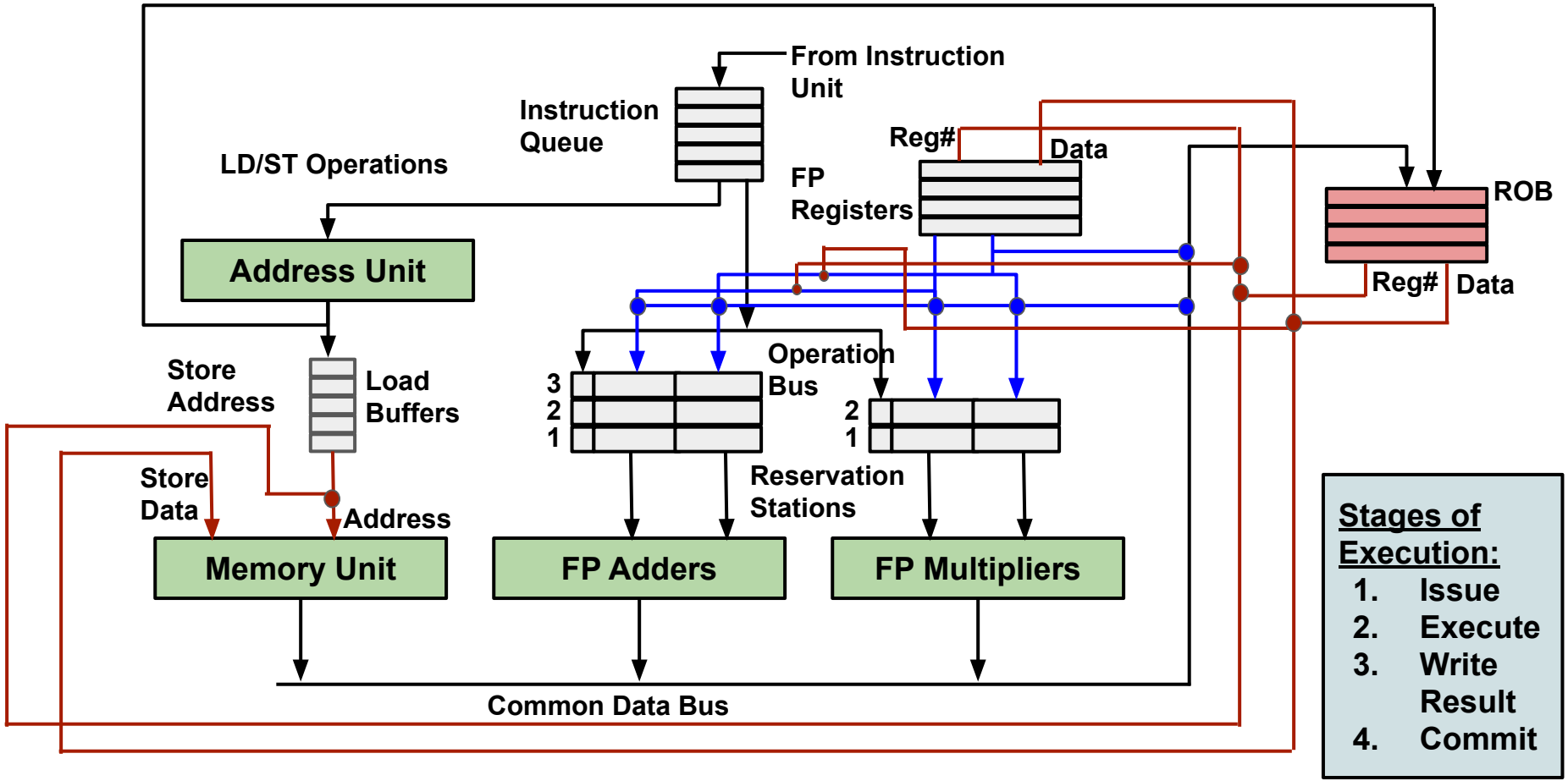
Indicates if the instruction is a branch

Branch Target Address/Register Number (After Evaluation)

Holds the value of the instruction result until the instruction commits

Indicates if the instruction completed execution, and the value is ready

Hardware Speculation

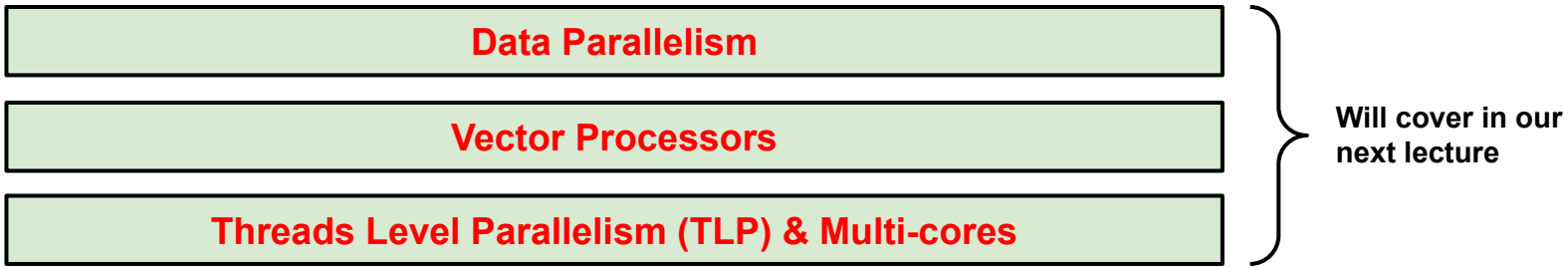


Stages of Execution:

1. Issue
2. Execute
3. Write Result
4. Commit

Summary

- Branch Prediction: Static vs. Dynamic
- Out-of-Order Execution
  - Tomasulo's Algorithm
- Re-order Buffers and Hardware Speculation



Thank
You